calculus: the Calculus program — user guide

calculus-ai999x 1.1.0, Rust build: command line, desktop app and browser. **Creator / engineer:** Ibnul Jaif Farabi · **Company:** AI999X · support@ai999x.com

Calculus covers limits, derivatives, integrals, series, Taylor polynomials, multivariable and vector calculus, special functions and numerical differentiation and integration. This guide is about `calculus`, the model as a self-contained program for Windows and Linux: 71 capabilities in 14 topics, answered on your own computer with no network, no Python and nothing else to install. The same compiled model runs in the AI999X desktop app and in the browser on the model's page (https://www.ai999x.com/models/calculus-ai999x/); the Python package (`pip install calculus-ai999x`) has its own guide. Every command output printed here is what the program printed.

Contents

- 1. Get it: desktop app, command line, browser, Python
- 2. Quick start
- 3. Writing inputs
- 4. Outcomes and exit codes
- 5. Scripts and agents: batch and JSON
- 6. Figures
- 7. In a web page or Node.js: WebAssembly
- 8. Every capability

1. Get it

Version 1.4.0 of the AI999X desktop app and of the command-line bundle. Checksums for every file: https://downloads.ai999x.com/desktop/1.4.0/SHA256SUMS; the newest version is always under https://downloads.ai999x.com/desktop/latest/. Full instructions: https://www.ai999x.com/desktop-cli/

what	system	download
Desktop app: Calculus	Windows x86-64	https://downloads.ai999x.com/desktop/1.4.0/ai999x-calculus-1.4.0-windows-x86_64.zip
Desktop app: Calculus	Linux x86-64	https://downloads.ai999x.com/desktop/1.4.0/ai999x-calculus-1.4.0-linux-x86_64.tar.gz
Desktop app: all models	Windows x86-64	https://downloads.ai999x.com/desktop/1.4.0/ai999x-1.4.0-windows-x86_64.zip
Desktop app: all models	Linux x86-64	https://downloads.ai999x.com/desktop/1.4.0/ai999x-1.4.0-linux-x86_64.tar.gz
Command line	Windows x86-64	https://downloads.ai999x.com/desktop/1.4.0/ai999x-cli-1.4.0-windows-x86_64.zip
Command line	Linux x86-64	https://downloads.ai999x.com/desktop/1.4.0/ai999x-cli-1.4.0-linux-x86_64.tar.gz

Desktop app

Calculus in its own app: search, questions in words, input forms, live figures and explanations. Windows: Download ai999x-calculus-1.4.0-windows-x86_64.zip, right-click it → Extract All, then open the folder and start ai999x-calculus.exe. If Windows SmartScreen asks, choose More info → Run anyway. Linux:

```
curl -LO https://downloads.ai999x.com/desktop/1.4.0/ai999x-calculus-1.4.0-linux-x86_64.tar.gz
curl -LO https://downloads.ai999x.com/desktop/1.4.0/SHA256SUMS
sha256sum -c --ignore-missing SHA256SUMS
tar xzf ai999x-calculus-1.4.0-linux-x86_64.tar.gz
./ai999x-calculus-1.4.0-linux-x86_64/ai999x-calculus
```

It opens on this model: type a question or pick a capability, fill in the form and press Compute. The AI999X desktop app with every model works the same way (choose the model first). Windows: Download ai999x-1.4.0-windows-x86_64.zip, right-click it → Extract All, then open the folder and start ai999x.exe. If Windows SmartScreen asks, choose More info → Run anyway. Linux:

```
curl -LO https://downloads.ai999x.com/desktop/1.4.0/ai999x-1.4.0-linux-x86_64.tar.gz
curl -LO https://downloads.ai999x.com/desktop/1.4.0/SHA256SUMS
sha256sum -c --ignore-missing SHA256SUMS
tar xzf ai999x-1.4.0-linux-x86_64.tar.gz
./ai999x-1.4.0-linux-x86_64/ai999x
```

Command line — Linux

The bundle holds one program per model; `calculus` is this one. The checksum line should print OK.

```
curl -LO https://downloads.ai999x.com/desktop/1.4.0/ai999x-cli-1.4.0-linux-x86_64.tar.gz
curl -LO https://downloads.ai999x.com/desktop/1.4.0/SHA256SUMS
sha256sum -c --ignore-missing SHA256SUMS
mkdir -p ~/.local/share && tar xzf ai999x-cli-1.4.0-linux-x86_64.tar.gz -C ~/.local/share
echo 'export PATH="$HOME/.local/share/ai999x-cli-1.4.0-linux-x86_64:$PATH"' >> ~/.bashrc && source
~/.bashrc
calculus --version
```

Command line — Windows (PowerShell)

The fourth line should print True.

```
$F = "ai999x-cli-1.4.0-windows-x86_64.zip"
Invoke-WebRequest "https://downloads.ai999x.com/desktop/1.4.0/$F" -OutFile $F
Invoke-WebRequest "https://downloads.ai999x.com/desktop/1.4.0/SHA256SUMS" -OutFile SHA256SUMS
(Get-FileHash $F -Algorithm SHA256).Hash.ToLower() -eq ((Select-String -SimpleMatch $F
SHA256SUMS).Line -split ' ')[0]
Expand-Archive $F -DestinationPath "$env:LOCALAPPDATA\AI999X" -Force
$D = "$env:LOCALAPPDATA\AI999X\ai999x-cli-1.4.0-windows-x86_64"; $env:Path += ";$D"
[Environment]::SetEnvironmentVariable("Path", [Environment]::GetEnvironmentVariable("Path", "User")
+ ";$D", "User")
calculus --version
```

In your browser

Open https://www.ai999x.com/models/calculus-ai999x/, choose a capability and press Compute in your browser: the same program runs on your device as WebAssembly; nothing is sent anywhere.

Python

```
pip install calculus-ai999x
```

2. Quick start

2.1 Check the install

Prints the model, its version and that this is the Rust build.

```
$ calculus --version
calculus-ai999x 1.0.3 (Rust build, ax-engine 0.1.0)
```

2.2 List the capabilities

Every capability the program answers: its id (what you pass to solve) and its name.

```
$ calculus list
arc_length                          Arc length of a graph
area_between_curves                 Area between two curves
area_enclosed_curves                Area enclosed by two curves
arithmetic_series_sum               Sum of an arithmetic series
average_rate_of_change              Average rate of change
bessel_j                            Bessel function of the first kind
beta_function                       Beta function
centroid_region                     Centroid of a region under a curve
… 56 more
```

2.3 What a capability needs

The equation or method, every input and output with its SI unit. Read this before solve: it tells you the input names.

```
$ calculus info limit_at_point
Limit at a point
  exact symbolic limit (series expansion / Gruntz algorithm) from both sides; refused if the one-
sided limits differ or are infinite
  in   f        the function f(x) (text) [1]
  in   a        the point x = a [1]
  out  limit    the limit of f(x) as x -> a [1]
```

2.4 Solve

Inputs as name=value, each with any unit of the right kind. The answer lists every output, the method used, each input converted to SI, and the conditions the answer assumes.

```
$ calculus solve limit_at_point a=2.0 "f=(x^2 - 4)/(x - 2)"
limit = 4
  by: Limit at a point  (exact symbolic limit (series expansion / Gruntz algorithm) from both
sides; refused if the one-sided limits differ or are infinite)
  a = 2   []
  f = (x^2 - 4)/(x - 2)
  assumes: the left and right limits exist, are finite and agree
```

2.5 Machine-readable answer

--json prints the same answer as one line of JSON (values in SI): for scripts and other programs.

```
$ calculus solve limit_at_point a=2.0 "f=(x^2 - 4)/(x - 2)" --json
{"outcome":"SOLVED","message":"","value":4.0,"unit":"","solved_for":"limit","si":
[["a",2.0]],"values":[["limit",4.0,""]],"target":"limit","phrase":"the limit of f(x) as x ->
a","assumptions":["the left and right limits exist, are finite and agree"],"name":"Limit at a
point","equation":"exact symbolic limit (series expansion / Gruntz algorithm) from both sides;
refused if the one-sided limits differ or are infinite"}
```

2.6 When an input is missing

No number is guessed: the answer names what is missing, shows a complete call, and the program exits with status 1.

```
$ calculus solve limit_at_point a=2.0
NEEDS_INPUT: limit_at_point needs f (the function f(x)). Example: solve('limit_at_point', f='(x^2 -
4)/(x - 2)', a=2).
```

2.7 Many questions at once

One JSON request per line in, one JSON answer per line out, in order: for scripts, notebooks and agents (the program starts once). In PowerShell: Get-Content cases.jsonl | calculus batch

`cases.jsonl`:

```
{"key": "limit_at_point", "inputs": {"a": 2.0, "f": "(x^2 - 4)/(x - 2)"}}
{"key": "extrema_on_interval", "inputs": {"a": 0.0, "b": 2.0, "f": "x^3 - 3x"}}
```

```
$ calculus batch < cases.jsonl
{"outcome":"SOLVED","message":"","value":4.0,"unit":"","solved_for":"limit","si":
[["a",2.0]],"values":[["limit",4.0,""]],"target":"limit","phrase":"the limit of f(x) as x ->
a","assumptions":["the left and right limits exist, are finite and agree"],"name":"Limit at a
point","equation":"exact symbolic limit (series expansion / Gruntz algorithm) from both sides;
refused if the one-sided limits differ or are infinite"}
{"outcome":"SOLVED","message":"","value":2.0,"unit":"","solved_for":"absolute_max","si":[["a",0.0],
["b",2.0]],"values":[["absolute_max",2.0,""],["absolute_min",-2.0,""],["critical_points",
[1.0],""]],"target":"absolute_max","phrase":"the largest value of f on [a, b]","assumptions":["a <
b","f is a polynomial with rational coefficients"],"name":"Absolute extrema on a closed
interval","equation":"exact real roots of f' in (a, b) (Sturm isolation), then f compared at the
critical points and both ends"}
```

3. Writing inputs

Inputs are written `name=value`, one per argument; `calculus info <capability>` lists the names. Quote an argument that holds a space or brackets (bash, zsh and PowerShell all accept double quotes). The answer always states each input as it was read, converted to SI, so a misread unit is visible at once.

kind	example	how it is read
a number with a unit	`"R=0.128 nm"`	any unit of the same kind; converted to SI
a number in SI	`R=1.28e-10`	no unit means the SI unit shown by info
a choice	`structure=FCC`	one of the values info lists
a list or vector	`"v=[1, 2, 3]" or "v=1, 2, 3"`	brackets keep it a list
a matrix	`"A=[[1, 2], [3, 4]]"`	one list per row
true / false	`inclusive=true`	for yes/no options

An equation can be solved for any of its quantities: give the others and add `--target <symbol>`. Without `--target`, the program solves for the quantity the capability is named after.

4. Outcomes and exit codes

The program never guesses a number: when it cannot answer, it says why and exits with status 1, so a script can tell an answer from a refusal without reading the text.

outcome	meaning	exit status
SOLVED	the answer: every output with its unit, the method, the inputs in SI, the conditions assumed	0
NEEDS_INPUT	an input is missing; the message names it and shows a complete call	1
INVALID_INPUT	a value or unit of the wrong kind, an unknown input name, or a value outside the conditions	1
AMBIGUOUS	the inputs allow two different answers; say which one is meant	1
OUT_OF_SCOPE	the model has no capability for this, or the result would not be meaningful	1
(usage)	a command the program does not know: it prints its usage	2

5. Scripts and agents: batch and JSON

`calculus solve ... --json` prints one answer as one line of JSON. `calculus batch` reads one request per line on standard input and writes one JSON answer per line, in order, starting the program once for any number of requests. A request is `{"key": "<capability>", "inputs": {"name": "value", ...}, "target": null}`. In PowerShell, pipe the file in: `Get-Content cases.jsonl | calculus batch`. `calculus catalog` prints every capability with its inputs as JSON, to build forms or tool definitions.

answer field	meaning
outcome	one of the outcomes above
message	why there is no answer (empty when SOLVED)
value, unit	the main result in SI units
values	every output: [symbol, value, unit] (procedures)
target	the quantity solved for
si	each numeric input converted to SI: [symbol, value]

answer field	meaning
assumptions	the conditions the answer relies on
name, equation	the capability and its equation or method

```
import json, subprocess

reqs = [{"key": "<capability>", "inputs": {"name": "value"}}]
out = subprocess.run(["calculus", "batch"], input="\n".join(map(json.dumps, reqs)),
                     capture_output=True, text=True, check=True).stdout
answers = [json.loads(line) for line in out.splitlines()]
```

6. Figures

`calculus figures <capability> name=value ... --out DIR` draws the live figures for those inputs and writes each as an SVG file into DIR (an existing folder); without `--out` the SVG text comes back as JSON. The desktop app and the model page draw the same figures as soon as you press Compute.

7. In a web page or Node.js: WebAssembly

The same model is a WebAssembly module, `calculus_ai999x.wasm` (https://www.ai999x.com/compute/rust/calculus_ai999x.wasm), with a small loader, `ax-wasm.js` (https://www.ai999x.com/compute/rust/ax-wasm.js). It runs in a browser, a Web Worker or Node.js, with no server.

```
<script src="ax-wasm.js"></script>
<script>
  AXWasm.load("calculus_ai999x.wasm").then(function (m) {
    var a = m.solve({key: "<capability>", inputs: {name: "value"}});
    console.log(a.outcome, a.values || a.value);
    console.log(m.catalog().capabilities.length + " capabilities");
  });
</script>
```

8. Every capability

By topic. For each: what it does, what it needs and gives, when it applies, and a command with what the program printed for it. `calculus info <id>` shows the same inputs and outputs in the terminal.

Derivatives (14)

Absolute extrema on a closed interval — `extrema_on_interval`

The absolute maximum and minimum values of a polynomial on [a, b] by the closed-interval method, with the critical points inside (a, b).

$$\max_{[a,b]} f = \max\{f(a), f(b), f(c) : f'(c) = 0\}$$

symbol		meaning	SI unit
`f`	in	the polynomial f(x)	—
`a`	in	left end	—
`b`	in	right end	—
`absolute_max`	out	the largest value of f on [a, b]	—
`absolute_min`	out	the smallest value	—
`critical_points`	out	critical points in (a, b), ascending	—

Applies when: a < b; f is a polynomial with rational coefficients.

```
$ calculus solve extrema_on_interval a=0 b=2 "f=x^3 - 3x"
absolute_max = 2
absolute_min = -2
critical_points = [1.0]
  by: Absolute extrema on a closed interval  (exact real roots of f' in (a, b) (Sturm isolation),
then f compared at the critical points and both ends)
  a = 0   []
  b = 2   []
  f = x^3 - 3x
  assumes: a < b; f is a polynomial with rational coefficients
```

What the worked example means: This calculation determines the highest and lowest values attained by the polynomial f(x) over the closed interval from a to b. The method identifies all critical points where the derivative vanishes within the open interval, then compares the function value at each such point against the values at the two endpoints. The largest of these comparisons yields the absolute maximum 2, while the smallest gives the absolute minimum -2. This approach relies on the Extreme Value Theorem, which guarantees that a continuous function on a closed bounded interval must attain both its global extrema.

Average rate of change — `average_rate_of_change`

The average rate of change (f(b) - f(a))/(b - a) of f over [a, b]: the slope of the secant line.

$$\frac{\Delta f}{\Delta x} = \frac{f(b) - f(a)}{b - a}$$

symbol		meaning	SI unit
`f`	in	the function f(x)	—
`a`	in	start of the interval	—
`b`	in	end of the interval	—
`average_rate`	out	(f(b) - f(a))/(b - a)	—
`fa`	out	f(a)	—
`fb`	out	f(b)	—

Applies when: a != b; f defined at a and b.

```
$ calculus solve average_rate_of_change a=1 b=3 "f=x^2"
average_rate = 4
fa = 1
fb = 9
  by: Average rate of change  (difference quotient of the two function values)
  a = 1   []
  b = 3   []
  f = x^2
  assumes: a != b; f defined at a and b
```

What the worked example means: This calculation determines the average rate of change for the function over the interval defined by the start point 1 and the end point 3. It is computed using the difference quotient, which takes the value of the function at the end point, 9, subtracts the value at the start point, 1, and divides this difference by the width of the interval. The resulting value, 4, represents the slope of the secant line connecting these two points on the graph, indicating the mean speed or change per unit interval.

Critical points and their classification — `critical_points_classify`

All critical points of a polynomial f(x) and which are local maxima or minima (first-derivative test).

$$f'(c) = 0,\ f' \text{ changes } + \to - \Rightarrow \text{local max}$$

symbol		meaning	SI unit
`f`	in	the polynomial f(x)	—
`critical_points`	out	real solutions of f'(x) = 0, ascending	—
`local_maxima`	out	critical points that are local maxima	—
`local_minima`	out	critical points that are local minima	—

Applies when: f is a polynomial of degree >= 2 with rational coefficients.

```
$ calculus solve critical_points_classify "f=x^3 - 3x"
critical_points = [-1.0,1.0]
local_maxima = [-1.0]
local_minima = [1.0]
  by: Critical points and their classification  (exact real roots of f' (Sturm isolation); the sign
of f' on each side decides max / min / neither)
  f = x^3 - 3x
  assumes: f is a polynomial of degree >= 2 with rational coefficients
```

What the worked example means: This calculation determines the locations where the slope of the polynomial x^3 - 3x is zero. Finding the exact real roots of the derivative identifies all critical points listed in ascending order as [-1, 1]. The sign change of the derivative around each point classifies them, resulting in the set of local maxima [-1] and the set of local minima [1].

Curvature of a graph — `curvature`

The curvature kappa = |f''| / (1 + f'^2)^(3/2) of y = f(x) at x = a, and the radius of curvature 1/kappa.

$$\kappa = \frac{|f''(a)|}{(1 + f'(a)^2)^{3/2}}$$

symbol		meaning	SI unit
`f`	in	the function f(x)	—
`a`	in	the point x = a	—
`curvature`	out	kappa at x = a	—
`radius_of_curvature`	out	1/kappa (absent where kappa = 0)	—

Applies when: f twice differentiable at a.

```
$ calculus solve curvature a=0 "f=x^2"
curvature = 2
radius_of_curvature = 0.5
  by: Curvature of a graph  (curvature formula of a graph from the exact first and second
derivatives)
  a = 0   []
  f = x^2
  assumes: f twice differentiable at a
```

What the worked example means: This calculation determines how sharply the graph of the function bends at the specific point defined by the input x-value. Using the exact first and second derivatives of the function, it computes the curvature value and the corresponding radius of the osculating circle. The result indicates that at this location, the curve has a curvature of 2 and fits within a circle of radius 0.5, providing a precise geometric measure of the local bend.

Derivative at a point — `derivative_at_point`

The derivative f'(a) of a function f(x) at x = a (the slope of its graph there).

$$f'(a) = \lim_{h \to 0} \frac{f(a + h) - f(a)}{h}$$

symbol		meaning	SI unit
`f`	in	the function f(x)	—
`a`	in	the point x = a	—
`derivative`	out	f'(a)	—
`derivative_expression`	out	f'(x) as an expression	—

Applies when: f is defined and differentiable at x = a.

```
$ calculus solve derivative_at_point a=2 "f=x^3 - 2x"
derivative = 10
derivative_expression = 3*x^2 - 2
  by: Derivative at a point  (exact symbolic differentiation (sum, product, quotient and chain
rules), then evaluation at x = a)
  a = 2   []
  f = x^3 - 2x
  assumes: f is defined and differentiable at x = a
```

What the worked example means: This calculation determines the instantaneous rate of change of the function f(x) at the specific input value a. Applying exact symbolic differentiation rules such as the sum, product, quotient, and chain rules gives the general expression for the slope, which is 3*x^2 - 2. This expression is then evaluated at the given point to find the precise numerical slope of the tangent line, resulting in 10. This value represents how steeply the curve rises or falls at that exact location.

Higher derivative at a point — `nth_derivative_at_point`

The n-th derivative f^(n)(a) of f(x) at x = a.

$$f^{(n)}(a) = \frac{d^n f}{dx^n}\Big|_{x=a}$$

symbol		meaning	SI unit
`f`	in	the function f(x)	—
`n`	in	the order of the derivative	—
`a`	in	the point x = a	—
`nth_derivative`	out	f^(n)(a)	—
`nth_derivative_expression`	out	f^(n)(x)	—

Applies when: 1 <= n <= 12; f is n times differentiable at x = a.

```
$ calculus solve nth_derivative_at_point a=1 "f=sin(2x)" n=4
nth_derivative = 14.54875883
nth_derivative_expression = 16*sin(2*x)
  by: Higher derivative at a point  (exact symbolic differentiation n times, then evaluation at x =
a)
  a = 1   []
  f = sin(2x)
  assumes: 1 <= n <= 12; f is n times differentiable at x = a
```

What the worked example means: This calculation determines the exact symbolic expression for the derivative of order 4 and then evaluates it at the specific input point 1. The resulting value, 14.5488, represents the rate of change of the (n-1)-th derivative at that location. This process relies on repeatedly applying differentiation rules to the original function until the desired order is reached, providing a precise mathematical snapshot of the function's higher-order behavior.

Implicit differentiation — `implicit_derivative`

The slope dy/dx of a curve given implicitly by F(x, y) = c, at a point on the curve.

$$\frac{dy}{dx} = -\frac{F_x}{F_y}$$

symbol		meaning	SI unit
`F`	in	the curve as an equation in x and y	—
`p`	in	the point (x, y) on the curve	—
`dy_dx`	out	dy/dx at the point	—

Applies when: the point lies on the curve; F_y != 0 there (no vertical tangent).

```
$ calculus solve implicit_derivative "F=x^2 + y^2 = 25" "p=[3, 4]"
dy_dx = -0.75
  by: Implicit differentiation  (dy/dx = -F_x/F_y with exact symbolic partial derivatives of F(x,
y) - c)
  F = x^2 + y^2 = 25
  assumes: the point lies on the curve; F_y != 0 there (no vertical tangent)
```

What the worked example means: This calculation determines the instantaneous rate of change of y with respect to x for a curve defined implicitly by an equation. Applying implicit differentiation gives the partial derivatives of the function defining the curve with respect to x and y separately. The resulting slope at the specific point [3, 4] is given as -0.75, which represents the steepness of the curve's tangent line at that location.

Inflection points — `inflection_points`

The points where a polynomial changes concavity (f'' changes sign).

$$f''(c) = 0 \text{ and } f'' \text{ changes sign at } c$$

symbol		meaning	SI unit
`f`	in	the polynomial f(x)	—
`inflection_points`	out	x-values of the inflection points, ascending	—

Applies when: f is a polynomial of degree >= 2 with rational coefficients.

```
$ calculus solve inflection_points "f=x^3 - 3x^2"
inflection_points = [1.0]
  by: Inflection points  (exact real roots of f''; kept only where f'' changes sign (odd
multiplicity))
  f = x^3 - 3x^2
  assumes: f is a polynomial of degree >= 2 with rational coefficients
```

What the worked example means: This calculation determines where the polynomial changes its concavity by finding the real roots of its second derivative. It filters these roots to keep only those where the sign of the second derivative actually changes, indicating a genuine switch from bending upward to downward or vice versa. The resulting x-values are listed in ascending order as [1], derived from the input function x^3 - 3x^2.

Linear approximation — `linear_approximation`

Estimate f(x1) by the tangent line at x = a: L(x1) = f(a) + f'(a)(x1 - a); with the actual value and error.

$$L(x) = f(a) + f'(a)\,(x - a)$$

symbol		meaning	SI unit
`f`	in	the function f(x)	—
`a`	in	the base point a	—
`x1`	in	the point to estimate	—
`approximation`	out	L(x1)	—
`actual`	out	f(x1)	—
`error`	out	f(x1) - L(x1)	—
`slope`	out	f'(a)	—

Applies when: f differentiable at a.

```
$ calculus solve linear_approximation a=4 "f=sqrt(x)" x1=4.1
approximation = 2.025
actual = 2.024845673
error = -0.0001543268683
slope = 0.25
  by: Linear approximation  (linearization (tangent line) at the base point)
  a = 4   []
  x1 = 4.1   []
  f = sqrt(x)
  assumes: f differentiable at a
```

What the worked example means: This calculation estimates the function's value at a specific target point using the tangent line derived from the base point. Applying the formula L(x) = f(a) + f'(a)(x-a) shows that the slope of the tangent is 0.25. Consequently, the approximated value is 2.025, while the true function value is 2.02485. The difference between these two, known as the error, is -0.000154327, indicating how closely the straight-line model matches the curve at this location.

Mean value theorem — `mean_value_theorem`

The secant slope of a polynomial on [a, b] and every c in (a, b) where f'(c) equals it.

$$f'(c) = \frac{f(b) - f(a)}{b - a}$$

symbol		meaning	SI unit
`f`	in	the polynomial f(x)	—
`a`	in	left end	—
`b`	in	right end	—
`secant_slope`	out	(f(b) - f(a))/(b - a)	—
`c_values`	out	all c in (a, b) with f'(c) = the slope	—

Applies when: a < b; f a non-linear polynomial with rational coefficients.

```
$ calculus solve mean_value_theorem a=0 b=3 "f=x^3"
secant_slope = 9
c_values = [1.7320508075688772]
  by: Mean value theorem  (exact secant slope, then the exact real roots of f'(x) - slope in (a,
b))
  a = 0   []
  b = 3   []
  f = x^3
  assumes: a < b; f a non-linear polynomial with rational coefficients
```

What the worked example means: This calculation determines the average rate of change of the polynomial f over the interval from a to b, resulting in a secant slope of 9. It then identifies all values c within that open interval where the instantaneous rate of change equals this average, yielding [1.73205]. This illustrates the Mean Value Theorem by showing specific locations where the tangent line is parallel to the secant line connecting the endpoints.

Normal line — `normal_line`

The normal line (perpendicular to the tangent) to y = f(x) at x = a: slope -1/f'(a).

$$y = f(a) - \frac{1}{f'(a)}\,(x - a)$$

symbol		meaning	SI unit
`f`	in	the function f(x)	—
`a`	in	the point x = a	—
`slope`	out	slope -1/f'(a)	—
`intercept`	out	y-intercept	—
`y0`	out	f(a)	—

Applies when: f'(a) != 0 (otherwise the normal is vertical).

```
$ calculus solve normal_line a=1 "f=x^2"
slope = -0.5
intercept = 1.5
y0 = 1
  by: Normal line  (perpendicular slope -1/m through (a, f(a)))
  a = 1   []
  f = x^2
  assumes: f'(a) != 0 (otherwise the normal is vertical)
```

What the worked example means: This calculation determines the equation of the normal line to the curve at the specific x-value 1. The normal is perpendicular to the tangent, so its slope is the negative reciprocal of the derivative, resulting in a value of -0.5. The line intersects the y-axis at 1.5, providing a complete linear description of the orthogonal direction at that point.

Second derivative at a point — `second_derivative_at_point`

The second derivative f''(a) of f(x) at x = a (concavity: positive means concave up).

$$f''(a) = \frac{d^2 f}{dx^2}\Big|_{x=a}$$

symbol		meaning	SI unit
`f`	in	the function f(x)	—
`a`	in	the point x = a	—
`second_derivative`	out	f''(a)	—
`second_derivative_expression`	out	f''(x)	—

Applies when: f is twice differentiable at x = a.

```
$ calculus solve second_derivative_at_point a=1 "f=x^4 - 3x^2"
second_derivative = 6
second_derivative_expression = 12*x^2 - 6
  by: Second derivative at a point  (exact symbolic differentiation twice, then evaluation at x =
a)
  a = 1   []
  f = x^4 - 3x^2
  assumes: f is twice differentiable at x = a
```

What the worked example means: This calculation finds the second derivative of the function f(x) evaluated at the point x equals a. Differentiating the function twice symbolically shows how the slope of the tangent line changes at that specific location. The resulting value, 6, describes the curvature; a positive value suggests the graph bends upward like a cup, while a negative value indicates it bends downward like a cap. The symbolic expression for this rate of change across all x is given as 6*(2*x^2 - 1).

Tangent line — `tangent_line`

The tangent line y = m*x + b to y = f(x) at x = a: slope m = f'(a), intercept b = f(a) - a*f'(a).

$$y = f(a) + f'(a)\,(x - a)$$

symbol		meaning	SI unit
`f`	in	the function f(x)	—
`a`	in	the point of tangency x = a	—
`slope`	out	slope m = f'(a)	—
`intercept`	out	y-intercept b	—
`y0`	out	f(a)	—
`equation`	out	the line as y = m*x + b	—

Applies when: f is differentiable at x = a.

```
$ calculus solve tangent_line a=2 "f=x^2 + 1"
slope = 4
intercept = -3
y0 = 5
equation = y = 4*x - 3
  by: Tangent line  (point-slope form through (a, f(a)) with the exact derivative)
  a = 2   []
  f = x^2 + 1
  assumes: f is differentiable at x = a
```

What the worked example means: This calculation determines the equation of the tangent line to the function f at the specific point where x equals 2. The process involves evaluating the function to find the y-coordinate 5 and computing the derivative to obtain the instantaneous rate of change, which serves as the slope 4. Using these values, the linear equation is constructed in the form y equals m times x plus b, yielding a final intercept of -3. This result represents the best linear approximation of the curve at that exact location, providing a clear geometric interpretation of the local behavior.

Velocity and acceleration from position — `velocity_acceleration`

Velocity v = s'(t) and acceleration a = s''(t) at time t0 from a position function s(t).

$$v = \frac{ds}{dt}, \quad a = \frac{d^2s}{dt^2}$$

symbol		meaning	SI unit
`s`	in	the position s(t)	—
`t0`	in	the time t0	—
`velocity`	out	v(t0) = s'(t0)	—
`acceleration`	out	a(t0) = s''(t0)	—
`speed`	out	|v(t0)|	—
`position`	out	s(t0)	—

Applies when: s twice differentiable at t0.

```
$ calculus solve velocity_acceleration "s=t^3 - 6t^2 + 9t" t0=2
velocity = -3
acceleration = 0
speed = 3
position = 2
  by: Velocity and acceleration from position  (exact differentiation of the position once and
twice)
  t0 = 2   []
  s = t^3 - 6t^2 + 9t
  assumes: s twice differentiable at t0
```

What the worked example means: This calculation determines the state of motion at a specific instant by differentiating the position function. Given the position s(t), its first derivative gives the instantaneous velocity and its second derivative the acceleration. At the specified time 2, the object's location is recorded as 2. The rate of change of this position, known as velocity, is calculated as -3. Consequently, the speed, which is the

magnitude of velocity regardless of direction, is 3. The rate of change of velocity, or acceleration, is found to be 0. These values describe how the object moves through space at that precise moment.

Fourier series (1)

Fourier coefficients — `fourier_coefficients`

The Fourier coefficients a_0, a_k and b_k of f on [-L, L] (series a_0/2 + sum a_k cos(k pi x/L) + b_k sin(k pi x/L)).

$$a_k = \frac{1}{L}\int_{-L}^{L} f(x)\cos\frac{k\pi x}{L}\,dx, \quad b_k = \frac{1}{L}\int_{-L}^{L} f(x)\sin\frac{k\pi x}{L}\,dx$$

symbol		meaning	SI unit
`f`	in	the function f(x)	—
`L`	in	the half-period L (> 0)	—
`k`	in	the order k	—
`a0`	out	a_0 = (1/L) integral of f	—
`a_k`	out	a_k	—
`b_k`	out	b_k	—

Applies when: L > 0; f continuous on [-L, L]; 1 <= k <= 50.

```
$ calculus solve fourier_coefficients L=3.141592654 f=x k=2
a0 = 0
a_k = -2.806286047e-39
b_k = -1
  by: Fourier coefficients  (tanh-sinh quadrature on the k half-wave subintervals at 20 digits)
  L = 3.141592654   []
  f = x
  assumes: L > 0; f continuous on [-L, L]; 1 <= k <= 50
```

In the worked example, L = 3.141592654, f = x, k = 2 go through the equation above and give a0 = 0, a_k = 0, b_k = -1. It applies only when L > 0; f continuous on [-L, L]; 1 <= k <= 50: outside that it says what is wrong instead of returning a number.

Integrals (14)

Arc length of a graph — `arc_length`

The length of the curve y = f(x) from x = a to x = b.

$$L = \int_a^b \sqrt{1 + f'(x)^2}\,dx$$

symbol		meaning	SI unit
`f`	in	the curve f(x)	—
`a`	in	left end	—

symbol		meaning	SI unit
`b`	in	right end	—
`arc_length`	out	the length of the curve	—

Applies when: a < b; f' continuous on [a, b].

```
$ calculus solve arc_length a=0 b=1 "f=x^2"
arc_length = 1.478942858
  by: Arc length of a graph  (arc-length integral of sqrt(1 + f'(x)^2) with the exact derivative,
by high-precision quadrature)
  a = 0   []
  b = 1   []
  f = x^2
  assumes: a < b; f' continuous on [a, b]
```

What the worked example means: This calculation determines the total distance along the curved path defined by the function f(x) between the left end a and the right end b. Integrating the square root of one plus the squared derivative sums infinitesimal segments to find the precise geometric length. The resulting value, 1.47894, represents this physical extent for the specific interval from 0 to 1.

Area between two curves — `area_between_curves`

The area between the graphs of two polynomials y = f(x) and y = g(x) over [a, b], counting every piece as positive (the curves may cross).

$$A = \int_a^b |f(x) - g(x)|\,dx$$

symbol		meaning	SI unit
`f`	in	the first curve f(x) (a polynomial)	—
`g`	in	the second curve g(x) (a polynomial)	—
`a`	in	left end	—
`b`	in	right end	—
`area`	out	the area between the curves	—
`net_integral`	out	integral of f - g (signed)	—
`crossings`	out	x where the curves cross inside (a, b)	—

Applies when: a < b; f and g polynomials with rational coefficients.

```
$ calculus solve area_between_curves a=0 b=2 "f=x^2" g=x
area = 1
net_integral = 0.6666666667
crossings = [1.0]
  by: Area between two curves  (exact integration of |f - g|, split at the exact real roots of f -
g)
  a = 0   []
  b = 2   []
  g = x
  f = x^2
  assumes: a < b; f and g polynomials with rational coefficients
```

What the worked example means: This calculation determines the total physical space enclosed between two polynomial graphs, y equals f of x and y equals g of x, over the interval from a to b. Integrating the absolute difference of the functions ensures that regions where the curves cross are counted positively rather than canceling out. The result, 1, represents this accumulated geometric magnitude, distinct from the signed value 0.666667 which accounts for direction.

Area enclosed by two curves — `area_enclosed_curves`

The area of the region enclosed by two polynomial curves y = f(x) and y = g(x), between their first and last intersection.

$$A = \int_{x_1}^{x_n} |f(x) - g(x)|\,dx$$

symbol		meaning	SI unit
`f`	in	the first curve f(x) (a polynomial)	—
`g`	in	the second curve g(x) (a polynomial)	—
`area`	out	the enclosed area	—
`x_left`	out	first intersection	—
`x_right`	out	last intersection	—
`intersections`	out	all intersection x-values	—

Applies when: f and g polynomials with rational coefficients meeting in at least two points.

```
$ calculus solve area_enclosed_curves "f=x^2" g=x
area = 0.1666666667
x_left = 0
x_right = 1
intersections = [0.0,1.0]
  by: Area enclosed by two curves  (exact real roots of f - g, then exact integration of |f - g|
between consecutive roots)
  f = x^2
  g = x
  assumes: f and g polynomials with rational coefficients meeting in at least two points
```

What the worked example means: This calculation determines the total geometric area trapped between two polynomial curves, defined as f and g. By finding all points where these functions intersect, the method

identifies the horizontal boundaries of the enclosed region. The final result, 0.166667, represents the sum of the absolute differences between the upper and lower curves across this interval, providing a precise measure of the space contained within their loops.

Average value of a function — `average_value`

The average value (1/(b - a)) * integral of f from a to b of a function on [a, b].

$$f_{\text{avg}} = \frac{1}{b-a}\int_a^b f(x)\,dx$$

symbol		meaning	SI unit
`f`	in	the function f(x)	—
`a`	in	start of the interval	—
`b`	in	end of the interval	—
`average_value`	out	the average value of f on [a, b]	—
`integral`	out	the integral of f over [a, b]	—

Applies when: a < b; f continuous on [a, b].

Not in `calculus` 1.1.0 yet: use the Python package or the model's page in the browser.

What the worked example means: This calculation determines the mean height of a continuous function across a specific interval. Computing the definite integral, which represents the total accumulated area under the curve as 9, and dividing it by the width of the interval defined by the start point 0 and end point 3 gives the average value 3. This result indicates the constant level that would produce the same total accumulation as the varying function over the given domain.

Centroid of a region under a curve — `centroid_region`

The area and centroid (x_bar, y_bar) of the region between y = f(x) >= 0 (a polynomial) and the x-axis for a <= x <= b.

$$\bar{x} = \frac{1}{A}\int_a^b x f(x)\,dx, \quad \bar{y} = \frac{1}{2A}\int_a^b f(x)^2\,dx$$

symbol		meaning	SI unit
`f`	in	the curve f(x) (a polynomial)	—
`a`	in	left end	—
`b`	in	right end	—
`area`	out	area of the region	—
`x_bar`	out	x-coordinate of the centroid	—
`y_bar`	out	y-coordinate of the centroid	—

Applies when: a < b; f(x) >= 0 on [a, b] (checked exactly).

```
$ calculus solve centroid_region a=0 b=1 "f=x^2"
area = 0.3333333333
x_bar = 0.75
y_bar = 0.3
  by: Centroid of a region under a curve  (exact moments of the region: x_bar = (1/A) * integral of
x f, y_bar = (1/2A) * integral of f^2)
  a = 0   []
  b = 1   []
  f = x^2
  assumes: a < b; f(x) >= 0 on [a, b] (checked exactly)
```

What the worked example means: This calculation determines the geometric center and total size of the region bounded by the polynomial curve f(x) and the x-axis between the limits a and b. The area is found by integrating the function over the interval, while the centroid coordinates are derived from the first moments of that area. Specifically, the x-coordinate of the balance point is the integral of x times f(x) divided by the total area, and the y-coordinate is half the integral of the square of f(x) divided by the area. For the given inputs, the resulting area is 0.333333, with the centroid located at horizontal position 0.75 and vertical position 0.3. This point represents where the shape would balance perfectly if it were made of a uniform material.

Definite integral — `definite_integral`

The definite integral of f(x) from x = a to x = b (net signed area).

$$\int_a^b f(x)\,dx$$

symbol		meaning	SI unit
`f`	in	the integrand f(x)	—
`a`	in	lower limit	—
`b`	in	upper limit	—
`integral`	out	the value of the integral	—

Applies when: f real and continuous on the closed interval between a and b.

```
$ calculus solve definite_integral a=0 b=3 "f=x^2"
integral = 9
  by: Definite integral  (high-precision tanh-sinh quadrature (30 digits) after an exact continuity
check on [a, b])
  a = 0   []
  b = 3   []
  f = x^2
  assumes: f real and continuous on the closed interval between a and b
```

What the worked example means: This calculation determines the net signed area under the curve of the integrand between the specified limits. The method verifies that the function is continuous over the closed interval before applying high-precision numerical quadrature. The resulting value, 9, represents the accumulated quantity defined by the integral of f(x) with respect to x from a to b.

Derivative of an integral (fundamental theorem) — `ftc_derivative`

d/dx of the integral of f(t) from a to g(x), evaluated at x = x0 (first fundamental theorem with the chain rule).

$$\frac{d}{dx}\int_a^{g(x)} f(t)\,dt = f(g(x))\,g'(x)$$

symbol		meaning	SI unit
`f`	in	the integrand f(t)	—
`a`	in	the constant lower limit	—
`g`	in	the upper limit g(x)	—
`x0`	in	the point x = x0	—
`derivative`	out	d/dx of the integral at x0	—

Applies when: f continuous between a and g(x0); g differentiable at x0.

```
$ calculus solve ftc_derivative a=0 "f=sin(t)" "g=x^2" x0=1
derivative = 1.68294197
  by: Derivative of an integral (fundamental theorem)  (f(g(x0)) * g'(x0) with the exact derivative
of g)
  a = 0   []
  x0 = 1   []
  g = x^2
  f = sin(t)
  assumes: f continuous between a and g(x0); g differentiable at x0
```

What the worked example means: This calculation finds how fast the area under the curve f grows when the upper limit g moves past x0. Applying the fundamental theorem and chain rule multiplies the value of the integrand at the current upper limit by the speed at which that limit changes. The result, 1.68294, represents this instantaneous growth rate, showing that only the behavior at the upper boundary affects the derivative, while the constant lower limit contributes nothing to the change.

Improper integral to infinity — `improper_integral`

The improper integral of f(x) from x = a to infinity, when it converges.

$$\int_a^{\infty} f(x)\,dx = \lim_{R\to\infty}\int_a^{R} f(x)\,dx$$

symbol		meaning	SI unit
`f`	in	the integrand f(x)	—
`a`	in	lower limit	—
`integral`	out	the value of the improper integral	—

Applies when: f continuous on [a, infinity); x^1.01 * f(x) has a finite limit (comparison test), so the integral converges absolutely.

Not in `calculus` 1.1.0 yet: use the Python package or the model's page in the browser.

What the worked example means: This calculation determines the area under the curve of the integrand from the specified lower limit extending infinitely outward. The method first verifies convergence by comparing the decay rate of the function against a power law, ensuring that the product of the variable raised to one point zero one and the function approaches a finite limit. Once convergence is established, the numerical value is computed using high-precision quadrature techniques on the unbounded domain. The resulting value, 1, represents the total accumulated quantity over this infinite range.

Left Riemann sum — `left_riemann_sum`

The left-endpoint Riemann sum of f on [a, b] with n equal subintervals.

$$L_n = h \sum_{k=0}^{n-1} f(a + kh), \quad h = \frac{b-a}{n}$$

symbol		meaning	SI unit
`f`	in	the function f(x)	—
`a`	in	left end	—
`b`	in	right end	—
`n`	in	number of equal subintervals	—
`approximation`	out	the left Riemann sum	—

Applies when: a < b; 1 <= n <= 100000; f defined at the sample points.

```
$ calculus solve left_riemann_sum a=0 b=2 "f=x^2" n=4
approximation = 1.75
  by: Left Riemann sum  (h times the sum of f at the left endpoints x_k = a + k h, k = 0..n-1 (h =
(b - a)/n))
  a = 0   []
  b = 2   []
  f = x^2
  assumes: a < b; 1 <= n <= 100000; f defined at the sample points
```

What the worked example means: This calculation determines the area under the curve by summing the heights of rectangles based on the function value at the left edge of each equal-width strip. Using 0 as the start and 2 as the end, with 4 intervals, the resulting estimate for the integral is 1.75. This method provides a lower or upper bound depending on whether the function is increasing or decreasing across the interval.

Right Riemann sum — `right_riemann_sum`

The right-endpoint Riemann sum of f on [a, b] with n equal subintervals.

$$R_n = h \sum_{k=1}^{n} f(a + kh), \quad h = \frac{b-a}{n}$$

symbol		meaning	SI unit
`f`	in	the function f(x)	—
`a`	in	left end	—

symbol		meaning	SI unit
`b`	in	right end	—
`n`	in	number of equal subintervals	—
`approximation`	out	the right Riemann sum	—

Applies when: a < b; 1 <= n <= 100000; f defined at the sample points.

```
$ calculus solve right_riemann_sum a=0 b=2 "f=x^2" n=4
approximation = 3.75
  by: Right Riemann sum  (h times the sum of f at the right endpoints x_k = a + k h, k = 1..n)
  a = 0   []
  b = 2   []
  f = x^2
  assumes: a < b; 1 <= n <= 100000; f defined at the sample points
```

What the worked example means: This calculation determines the right-endpoint Riemann sum for the function f on the interval from a to b using n equal subintervals. The width of each rectangle, h, is derived from the difference between the bounds divided by the count of subintervals. Evaluating the function at the right boundary of each segment and summing these heights multiplied by the width gives an estimate of the total area under the curve. For the given inputs, this method yields an approximation value of 3.75, representing the discrete accumulation of the function's values across the specified domain.

Surface area of revolution — `surface_area_revolution`

The area of the surface formed by rotating y = f(x), a <= x <= b, about the x-axis.

$$S = 2\pi \int_a^b |f(x)| \sqrt{1 + f'(x)^2}\, dx$$

symbol		meaning	SI unit
`f`	in	the curve f(x)	—
`a`	in	left end	—
`b`	in	right end	—
`surface_area`	out	the area of the surface of revolution	—

Applies when: a < b; f' continuous on [a, b].

```
$ calculus solve surface_area_revolution a=0 b=1 f=x
surface_area = 4.442882938
  by: Surface area of revolution  (2*pi times the integral of |f| * sqrt(1 + f'^2) (split where f
changes sign), high-precision quadrature)
  a = 0   []
  b = 1   []
  f = x
  assumes: a < b; f' continuous on [a, b]
```

What the worked example means: This calculation determines the total surface area created by rotating a curve around the x-axis. By integrating the product of the distance from the axis and the arc length element, it

accounts for the geometry of the resulting shape. The computed value of 4.44288 represents the exact area of this generated surface based on the function f(x) between limits a and b.

Volume of revolution (disk method) — `disk_volume`

The volume of the solid formed by rotating y = f(x), a <= x <= b, about the x-axis.

$$V = \pi \int_a^b f(x)^2 \, dx$$

symbol		meaning	SI unit
`f`	in	the curve f(x)	—
`a`	in	left end	—
`b`	in	right end	—
`volume`	out	the volume of the solid	—

Applies when: a < b; f continuous on [a, b].

```
$ calculus solve disk_volume a=0 b=1 "f=x^2"
volume = 0.6283185307
  by: Volume of revolution (disk method)  (disk method: pi times the integral of f^2, by high-
precision quadrature)
  a = 0   []
  b = 1   []
  f = x^2
  assumes: a < b; f continuous on [a, b]
```

What the worked example means: This calculation determines the volume of a solid of revolution using the disk method. Integrating the square of the function x^2 over the interval defined by 0 and 1, and multiplying by pi gives the total space occupied by the shape. The resulting value, 0.628319, represents the physical volume enclosed when the curve is spun around the horizontal axis.

Volume of revolution (shell method) — `shell_volume`

The volume swept when the region under y = f(x) >= 0 (a polynomial), a <= x <= b with a >= 0, is rotated about the y-axis.

$$V = 2\pi \int_a^b x \, f(x) \, dx$$

symbol		meaning	SI unit
`f`	in	the curve f(x) (a polynomial)	—
`a`	in	left end (>= 0)	—
`b`	in	right end	—
`volume`	out	the volume of the solid	—

Applies when: 0 <= a < b; f(x) >= 0 on [a, b] (checked exactly).

```
$ calculus solve shell_volume a=0 b=2 "f=x^2"
volume = 25.13274123
  by: Volume of revolution (shell method)  (cylindrical shells, integrated exactly)
  a = 0   []
  b = 2   []
  f = x^2
  assumes: 0 <= a < b; f(x) >= 0 on [a, b] (checked exactly)
```

What the worked example means: This calculation determines the volume of a three-dimensional solid formed by rotating a specific two-dimensional region around the vertical axis. The region is defined by the curve x^2, bounded horizontally between x equals 0 and x equals 2. By applying the cylindrical shells method, which sums the volumes of thin concentric tubes, the total space occupied by the rotated shape is found to be 25.1327. This result represents the exact capacity of the solid under the assumption that the function remains non-negative throughout the specified interval.

Volume of revolution (washer method) — `washer_volume`

The volume swept when the region between an outer curve y = f(x) and an inner curve y = g(x) (polynomials, f >= g >= 0) on [a, b] is rotated about the x-axis.

$$V = \pi \int_a^b \left(f(x)^2 - g(x)^2 \right) dx$$

symbol		meaning	SI unit
`f`	in	outer radius f(x) (a polynomial)	—
`g`	in	inner radius g(x) (a polynomial)	—
`a`	in	left end	—
`b`	in	right end	—
`volume`	out	the volume of the solid	—

Applies when: a < b; f >= g >= 0 on [a, b] (checked exactly); polynomials.

```
$ calculus solve washer_volume a=0 b=1 "f=x + 2" g=1
volume = 16.75516082
  by: Volume of revolution (washer method)  (washer method, integrated exactly)
  a = 0   []
  b = 1   []
  g = 1
  f = x + 2
  assumes: a < b; f >= g >= 0 on [a, b] (checked exactly); polynomials
```

What the worked example means: This calculation determines the volume of a hollow solid created by rotating the region between an outer curve and an inner curve around the x-axis. Using the washer method, it integrates the difference between the squares of the outer radius x + 2 and the inner radius 1 from the left end 0 to the right end 1. The resulting volume 16.7552 represents the total space occupied by this rotational shape.

Limits (2)

Limit at a point — `limit_at_point`

The two-sided limit of f(x) as x approaches a (including removable 0/0 forms such as sin(x)/x).

$$\lim_{x \to a} f(x)$$

symbol		meaning	SI unit
`f`	in	the function f(x)	—
`a`	in	the point x = a	—
`limit`	out	the limit of f(x) as x -> a	—

Applies when: the left and right limits exist, are finite and agree.

```
$ calculus solve limit_at_point a=2 "f=(x^2 - 4)/(x - 2)"
limit = 4
  by: Limit at a point  (exact symbolic limit (series expansion / Gruntz algorithm) from both
sides; refused if the one-sided limits differ or are infinite)
  a = 2   []
  f = (x^2 - 4)/(x - 2)
  assumes: the left and right limits exist, are finite and agree
```

What the worked example means: This calculation determines the value that the function (x^2 - 4)/(x - 2) approaches as its input variable gets arbitrarily close to the point 2. The result, 4, represents this two-sided limit. This means that whether approaching from values smaller than 2 or larger than 2, the function settles on the same finite number 4. If the left and right limits differed or were infinite, no single value would exist.

Limit at infinity — `limit_at_infinity`

The limit of f(x) as x grows without bound (horizontal asymptote, end behaviour).

$$\lim_{x \to \infty} f(x)$$

symbol		meaning	SI unit
`f`	in	the function f(x)	—
`limit`	out	the limit of f(x) as x -> infinity	—

Applies when: the limit exists and is finite.

```
$ calculus solve limit_at_infinity "f=(3x^2 + 1)/(x^2 - 5)"
limit = 3
  by: Limit at infinity  (exact symbolic limit at +infinity (Gruntz algorithm); refused if infinite
or oscillating)
  f = (3x^2 + 1)/(x^2 - 5)
  assumes: the limit exists and is finite
```

What the worked example means: This calculation determines the horizontal asymptote of the function (3x^2 + 1)/(x^2 - 5) by finding the value it approaches as the input grows without bound. The result, 3, represents the stable end behavior of the function at large positive values.

Multiple integrals (4)

Double integral over a rectangle — `double_integral_rectangle`

The double integral of f(x, y) over the rectangle [a, b] x [c, d].

$$\int_a^b \int_c^d f(x, y)\, dy\, dx$$

symbol		meaning	SI unit
`f`	in	the integrand f(x, y)	—
`a`	in	x from	—
`b`	in	x to	—
`c`	in	y from	—
`d`	in	y to	—
`integral`	out	the value of the double integral	—

Applies when: a < b, c < d; f continuous on the rectangle.

```
$ calculus solve double_integral_rectangle a=0 b=2 c=0 d=3 "f=x*y"
integral = 9
  by: Double integral over a rectangle  (product Gauss-Legendre rules of increasing order (24, 40,
64 nodes per direction) until two agree to 1e-12)
  a = 0   []
  b = 2   []
  c = 0   []
  d = 3   []
  f = x*y
  assumes: a < b, c < d; f continuous on the rectangle
```

What the worked example means: This calculation determines the total accumulation of the integrand across the specified rectangular region. By applying high-precision numerical integration, it estimates the volume under the surface defined by the function above the domain bounded by x limits 0 and 2, and y limits 0 and 3. The resulting value, 9, represents this integrated quantity for the continuous function provided.

Double integral over a region — `double_integral_region`

The double integral of f(x, y) over the region a <= x <= b, g1(x) <= y <= g2(x).

$$\int_a^b \int_{g_1(x)}^{g_2(x)} f(x, y)\, dy\, dx$$

symbol		meaning	SI unit
`f`	in	the integrand f(x, y)	—

symbol		meaning	SI unit
`a`	in	x from	—
`b`	in	x to	—
`g1`	in	the lower curve y = g1(x)	—
`g2`	in	the upper curve y = g2(x)	—
`integral`	out	the value of the double integral	—

Applies when: a < b; g1 <= g2 on [a, b]; f, g1, g2 continuous.

```
$ calculus solve double_integral_region a=0 b=1 "f=x*y" "g1=x^2" g2=x
integral = 0.04166666667
  by: Double integral over a region  (iterated Gauss-Legendre rules of increasing order (inner rule
mapped to [g1(x), g2(x)]) until two agree)
  a = 0   []
  b = 1   []
  f = x*y
  g1 = x^2
  g2 = x
  assumes: a < b; g1 <= g2 on [a, b]; f, g1, g2 continuous
```

What the worked example means: This calculation determines the total accumulation of the function x*y across the two-dimensional region bounded horizontally between 0 and 1, and vertically between the lower curve x^2 and the upper curve x. The method uses iterated numerical integration, refining the approximation until stable results are obtained. The final value, 0.0416667, represents the net signed volume or mass over this specific domain, assuming the integrand and boundary functions are continuous within the specified limits.

Surface area of a graph — `surface_area_graph`

The area of the surface z = f(x, y) over the rectangle [a, b] x [c, d].

$$A = \iint_R \sqrt{1 + f_x^2 + f_y^2}\, dA$$

symbol		meaning	SI unit
`f`	in	the surface z = f(x, y)	—
`a`	in	x from	—
`b`	in	x to	—
`c`	in	y from	—
`d`	in	y to	—
`surface_area`	out	the surface area	—

Applies when: a < b, c < d; f continuously differentiable on the rectangle.

```
$ calculus solve surface_area_graph a=0 b=1 c=0 d=1 "f=x + y"
surface_area = 1.732050808
  by: Surface area of a graph  (exact partial derivatives; Gauss-Legendre product rules of sqrt(1 +
f_x^2 + f_y^2) until two orders agree)
  a = 0   []
  b = 1   []
  c = 0   []
  d = 1   []
  f = x + y
  assumes: a < b, c < d; f continuously differentiable on the rectangle
```

What the worked example means: This calculation determines the total area of the curved surface defined by z equals f(x, y) across the rectangular region from x equal to 0 to x equal to 1, and y equal to 0 to y equal to 1. The method computes exact partial derivatives and applies a numerical integration technique to evaluate the square root term involving these derivatives. The resulting value, 1.73205, represents the physical extent of this surface in three-dimensional space.

Triple integral over a box — `triple_integral_box`

The triple integral of f(x, y, z) over the box [x0, x1] x [y0, y1] x [z0, z1].

$$\iiint_B f(x,y,z)\,dV$$

symbol		meaning	SI unit
`f`	in	the integrand f(x, y, z)	—
`x0`	in	x from	—
`x1`	in	x to	—
`y0`	in	y from	—
`y1`	in	y to	—
`z0`	in	z from	—
`z1`	in	z to	—
`integral`	out	the value of the triple integral	—

Applies when: x0 < x1, y0 < y1, z0 < z1; f continuous on the box.

```
$ calculus solve triple_integral_box "f=x*y*z" x0=0 x1=1 y0=0 y1=2 z0=0 z1=3
integral = 4.5
  by: Triple integral over a box  (exact iterated integration for a polynomial integrand; otherwise
product Gauss-Legendre rules of increasing order until two agree)
  x0 = 0   []
  x1 = 1   []
  y0 = 0   []
  y1 = 2   []
  z0 = 0   []
  z1 = 3   []
  f = x*y*z
  assumes: x0 < x1, y0 < y1, z0 < z1; f continuous on the box
```

What the worked example means: This calculation determines the volume accumulation of the function x*y*z across a rectangular region. The integration bounds are set by the intervals for x, y, and z coordinates, specifically from 0 to 1, 0 to 2, and 0 to 3. For polynomial inputs, the result is exact; otherwise, it uses adaptive numerical quadrature until convergence. The final computed value, 4.5, represents the total quantity such as mass or charge within that specific spatial domain.

Multivariable calculus (6)

Directional derivative — `directional_derivative`

The rate of change of f at a point in the direction of a vector u (u is normalised to unit length).

$$D_{\mathbf{u}} f = \nabla f \cdot \frac{\mathbf{u}}{|\mathbf{u}|}$$

symbol		meaning	SI unit
`f`	in	the function f(x, y) or f(x, y, z)	—
`p`	in	the point	—
`u`	in	the direction vector	—
`directional_derivative`	out	D_u f at the point	—

Applies when: u != 0 with as many components as p; f differentiable at p.

```
$ calculus solve directional_derivative "f=x^2*y" "p=[1, 2]" "u=[3, 4]"
directional_derivative = 3.2
  by: Directional derivative  (gradient (exact partial derivatives) dotted with u/|u|)
  f = x^2*y
  assumes: u != 0 with as many components as p; f differentiable at p
```

What the worked example means: This calculation determines the instantaneous rate of change of the function f at the specific location [1, 2] along the path defined by the direction vector [3, 4]. The method involves computing the gradient of the function and taking its dot product with the unit vector derived from [3, 4]. The resulting value, 3.2, represents how steeply the surface rises or falls when moving in that particular direction. If this value is positive, the function increases; if negative, it decreases; and if zero, the function remains constant along that path.

Gradient — `gradient`

The gradient vector of f(x, y) or f(x, y, z) at a point and its length (the greatest rate of increase).

$$\nabla f = (f_x, f_y, f_z)$$

symbol		meaning	SI unit
`f`	in	the function f(x, y) or f(x, y, z)	—
`p`	in	the point (x, y) or (x, y, z)	—
`gradient`	out	the gradient vector	—
`magnitude`	out	its length	—

Applies when: f differentiable at the point.

```
$ calculus solve gradient "f=x^2 + y^2" "p=[1, 2]"
gradient = [2.0,4.0]
magnitude = 4.472135955
  by: Gradient  (exact symbolic partial derivatives at the point)
  f = x^2 + y^2
  assumes: f differentiable at the point
```

What the worked example means: The gradient vector [2, 4] is computed by taking exact symbolic partial derivatives of the function x^2 + y^2 and evaluating them at the point [1, 2]. This vector points in the direction where the function increases most rapidly. Its length, given as 4.47214, represents the maximum rate of increase at that specific location. If you move in the direction of this vector, the function value will rise faster than in any other direction.

Jacobian determinant — `jacobian_determinant`

The Jacobian determinant d(x, y)/d(u, v) of a transformation x = x(u, v), y = y(u, v) at a point.

$$\frac{\partial(x, y)}{\partial(u, v)} = x_u y_v - x_v y_u$$

symbol		meaning	SI unit
`xu`	in	x as a function of u and v	—
`yv`	in	y as a function of u and v	—
`p`	in	the point (u, v)	—
`jacobian`	out	the Jacobian determinant at the point	—

Applies when: x and y differentiable at the point.

```
$ calculus solve jacobian_determinant "p=[2, 0.5]" "xu=u*cos(v)" "yv=u*sin(v)"
jacobian = 2
  by: Jacobian determinant  (exact partial derivatives; x_u y_v - x_v y_u)
  xu = u*cos(v)
  yv = u*sin(v)
  assumes: x and y differentiable at the point
```

What the worked example means: The Jacobian determinant is computed for the transformation defined by x and y at the point [2, 0.5]. Using the exact partial derivatives, the result is 2. This value indicates how much area scales locally under this coordinate change. If the magnitude of 2 is greater than one, the transformation expands areas; if it is less than one, it contracts them.

Partial derivative — `partial_derivative`

The partial derivative of f(x, y) or f(x, y, z) with respect to one variable, at a point.

$$\frac{\partial f}{\partial x}(p)$$

symbol		meaning	SI unit
`f`	in	the function f(x, y) or f(x, y, z)	—
`var`	in	the variable	—
`p`	in	the point (x, y) or (x, y, z)	—
`partial`	out	the partial derivative at the point	—

Applies when: f differentiable at the point.

```
$ calculus solve partial_derivative "f=x^2*y + y^3" "p=[1, 2]" var=x
partial = 4
  by: Partial derivative  (exact symbolic partial differentiation, then evaluation at the point)
  var = x
  f = x^2*y + y^3
  assumes: f differentiable at the point
```

What the worked example means: This calculation determines the instantaneous rate of change of the multivariable function as one specific input varies, while all other inputs are held fixed. Symbolically differentiating the function and evaluating the result at the given coordinates gives the value 4, which represents the slope of the tangent line to the curve formed by intersecting the surface with a plane parallel to the axis of differentiation.

Second derivative test (two variables) — `hessian_classification`

Classifies a critical point of f(x, y) as a local minimum, local maximum or saddle point from the Hessian.

$$D = f_{xx} f_{yy} - f_{xy}^2$$

symbol		meaning	SI unit
`f`	in	the function f(x, y)	—
`p`	in	the critical point (x, y)	—
`fxx`	out	f_xx	—
`fxy`	out	f_xy	—
`fyy`	out	f_yy	—
`D`	out	D = f_xx f_yy - f_xy^2	—
`classification`	out	local minimum, local maximum or saddle point	—

Applies when: the gradient is zero at the point; D != 0 (D = 0 is inconclusive and refused).

```
$ calculus solve hessian_classification "f=x^2 + 3y^2 - 2x" "p=[1, 0]"
fxx = 2
fxy = 0
fyy = 6
D = 12
classification = local minimum
  by: Second derivative test (two variables)  (exact second partial derivatives at the point; D > 0
and f_xx > 0 minimum, D > 0 and f_xx < 0 maximum, D < 0 saddle)
  f = x^2 + 3y^2 - 2x
  assumes: the gradient is zero at the point; D != 0 (D = 0 is inconclusive and refused)
```

In the worked example, f = x^2 + 3y^2 - 2x, p = [1, 0] go through the equation above and give fxx = 2, fxy = 0, fyy = 6, D = 12. It applies only when the gradient is zero at the point; D != 0 (D = 0 is inconclusive and refused): outside that it says what is wrong instead of returning a number.

Tangent plane — `tangent_plane`

The tangent plane z = f_x x + f_y y + c to the surface z = f(x, y) at (x0, y0).

$$z = f(x_0, y_0) + f_x(x - x_0) + f_y(y - y_0)$$

symbol		meaning	SI unit
`f`	in	the surface z = f(x, y)	—
`p`	in	the point (x0, y0)	—
`z0`	out	f(x0, y0)	—
`fx`	out	f_x(x0, y0)	—
`fy`	out	f_y(x0, y0)	—
`c`	out	the constant term c	—

Applies when: f differentiable at (x0, y0).

```
$ calculus solve tangent_plane "f=x^2 + y^2" "p=[1, 2]"
z0 = 5
fx = 2
fy = 4
c = -5
  by: Tangent plane  (exact partial derivatives; z = z0 + f_x (x - x0) + f_y (y - y0) expanded)
  f = x^2 + y^2
  assumes: f differentiable at (x0, y0)
```

What the worked example means: This calculation finds the best linear approximation of the surface near a specific point. Using the partial derivatives evaluated at [1, 2] shows that the slope in the x direction is 2 and the slope in the y direction is 4. The resulting equation, with constant term -5, describes the flat plane that touches the curved surface exactly at the height 5. This plane serves as a simple model for complex behavior in the immediate vicinity of the chosen coordinates.

Numerical differentiation (1)

Finite-difference derivative — `numerical_derivative`

A forward, backward or central difference approximation of f'(a) with step h, with the exact derivative and the error.

$$f'(a) \approx \frac{f(a+h) - f(a-h)}{2h}$$

symbol		meaning	SI unit
`f`	in	the function f(x)	—
`a`	in	the point x = a	—
`h`	in	the step size h > 0	—
`method`	in	forward, backward or central difference	—
`approximation`	out	the difference quotient	—
`exact`	out	the exact f'(a)	—
`error`	out	approximation - exact	—

Applies when: h > 0; f defined at the points used.

```
$ calculus solve numerical_derivative a=1 "f=x^2" h=0.1 method=central
approximation = 2
exact = 2
error = 0
  by: Finite-difference derivative  (difference quotients (f(a+h)-f(a))/h, (f(a)-f(a-h))/h,
(f(a+h)-f(a-h))/(2h) at 30 digits; exact derivative symbolically)
  a = 1   []
  h = 0.1   []
  method = central
  f = x^2
  assumes: h > 0; f defined at the points used
```

What the worked example means: This calculation estimates the instantaneous rate of change of the function f at the point x equals 1 using a step size of h equals 0.1. The method employed is a central difference, which yields an approximate derivative of 2. This result is then compared against the symbolically derived exact derivative of 2 to determine the numerical error, which stands at 0. Such approximations are vital when analytical differentiation is impractical, allowing for the estimation of slopes through nearby function values.

Numerical integration (4)

Gauss-Legendre quadrature — `gauss_legendre`

The n-point Gauss-Legendre approximation of the integral of f on [a, b] (exact for polynomials of degree <= 2n - 1), with the exact value and the error.

$$\int_a^b f\,dx \approx \frac{b-a}{2} \sum_{i=1}^{n} w_i\, f\left(\frac{a+b}{2} + \frac{b-a}{2} x_i\right)$$

symbol		meaning	SI unit
`f`	in	the function f(x)	—
`a`	in	left end	—
`b`	in	right end	—
`n`	in	number of nodes	—
`approximation`	out	n-point Gauss-Legendre value	—
`exact`	out	the integral	—
`error`	out	approximation - exact	—

Applies when: a < b; 1 <= n <= 64; f continuous on [a, b].

Not in `calculus` 1.1.0 yet: use the Python package or the model's page in the browser.

In the worked example, a = 0, b = 2, f = x^3 + x^2, n = 2 go through the equation above and give approximation = 6.666666667, exact = 6.666666667, error = 0. It applies only when a < b; 1 <= n <= 64; f continuous on [a, b]: outside that it says what is wrong instead of returning a number.

Midpoint rule — `midpoint_rule`

The composite midpoint approximation of the integral of f on [a, b] with n subintervals, with the exact value and the error.

$$M_n = h \sum_{k=0}^{n-1} f\left(a + (k + \tfrac{1}{2})h\right)$$

symbol		meaning	SI unit
`f`	in	the function f(x)	—
`a`	in	left end	—
`b`	in	right end	—
`n`	in	number of equal subintervals	—
`approximation`	out	midpoint approximation M_n	—
`exact`	out	the integral	—
`error`	out	M_n - exact	—

Applies when: a < b; 1 <= n <= 100000; f continuous on [a, b].

```
$ calculus solve midpoint_rule a=0 b=2 "f=x^2" n=4
approximation = 2.625
exact = 2.666666667
error = -0.04166666667
  by: Midpoint rule  (composite midpoint rule; exact value by high-precision quadrature)
  a = 0   []
  b = 2   []
  f = x^2
  assumes: a < b; 1 <= n <= 100000; f continuous on [a, b]
```

What the worked example means: This calculation determines the numerical estimate of a definite integral using the composite midpoint rule. By dividing the interval from a to b into n equal subintervals, the method sums the areas of rectangles whose heights are evaluated at the center of each segment. The resulting approximation is 2.625. For comparison, high-precision quadrature yields an exact integral value of 2.66667, leading to a computed error of -0.0416667. This process assumes that the function f is continuous over the specified domain.

Simpson's rule — `simpson_rule`

The composite Simpson approximation of the integral of f on [a, b] with an even number n of subintervals, with the exact value and the error.

$$S_n = \frac{h}{3}\left[f(x_0) + 4\sum_{\text{odd } k} f(x_k) + 2\sum_{\text{even } k} f(x_k) + f(x_n)\right]$$

symbol		meaning	SI unit
`f`	in	the function f(x)	—
`a`	in	left end	—
`b`	in	right end	—
`n`	in	number of equal subintervals	—
`approximation`	out	Simpson approximation S_n	—
`exact`	out	the integral	—
`error`	out	S_n - exact	—

Applies when: a < b; n even, 2 <= n <= 100000; f continuous on [a, b].

```
$ calculus solve simpson_rule a=0 b=2 "f=x^4" n=4
approximation = 6.416666667
exact = 6.4
error = 0.01666666667
  by: Simpson's rule  (composite Simpson's 1/3 rule (weights 1, 4, 2, ..., 4, 1); exact value by
high-precision quadrature)
  a = 0   []
  b = 2   []
  f = x^4
  assumes: a < b; n even, 2 <= n <= 100000; f continuous on [a, b]
```

What the worked example means: This calculation applies the composite Simpson one-third rule to approximate the definite integral of a continuous function over a specified interval. By dividing the range into an even number of equal subintervals and fitting quadratic polynomials across pairs of these segments, the method achieves higher accuracy than simpler linear approximations. The resulting estimate, 6.41667, is derived from the function values at the nodes using weights of one, four, and two. This value is then contrasted with the high-precision exact integral, 6.4, to determine the absolute deviation, reported here as 0.0166667.

Trapezoidal rule — `trapezoid_rule`

The composite trapezoidal approximation of the integral of f on [a, b] with n subintervals, with the exact value and the error (approximation - exact).

$$T_n = \frac{h}{2}\left[f(x_0) + 2\sum_{k=1}^{n-1} f(x_k) + f(x_n)\right]$$

symbol		meaning	SI unit
`f`	in	the function f(x)	—
`a`	in	left end	—
`b`	in	right end	—
`n`	in	number of equal subintervals	—
`approximation`	out	trapezoidal approximation T_n	—
`exact`	out	the integral	—
`error`	out	T_n - exact	—

Applies when: a < b; 1 <= n <= 100000; f continuous on [a, b].

```
$ calculus solve trapezoid_rule a=0 b=2 "f=x^2" n=4
approximation = 2.75
exact = 2.666666667
error = 0.08333333333
  by: Trapezoidal rule  (composite trapezoidal rule; exact value by high-precision quadrature)
  a = 0   []
  b = 2   []
  f = x^2
  assumes: a < b; 1 <= n <= 100000; f continuous on [a, b]
```

What the worked example means: This calculation determines the definite integral of a function over an interval using the composite trapezoidal rule. The method divides the area under the curve into equal subintervals and approximates each segment as a trapezoid, summing these areas to produce the approximation value of 2.75. This result is compared against the high-precision exact integral of 2.66667 to quantify the numerical error, which is reported as 0.0833333. A smaller absolute error indicates that the chosen number of subintervals provides a closer match to the true area.

Numerical methods (1)

Newton's method — `newton_method`

n iterations of Newton's method x_{k+1} = x_k - f(x_k)/f'(x_k) for a root of f, from x_0.

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$$

symbol		meaning	SI unit
`f`	in	the function f(x)	—
`x0`	in	the starting value x_0	—

symbol		meaning	SI unit
`n`	in	number of iterations	—
`x_n`	out	the n-th iterate	—
`f_x_n`	out	f(x_n)	—
`iterates`	out	x_1 .. x_n	—

Applies when: f'(x_k) != 0 at every iterate; 1 <= n <= 50.

```
$ calculus solve newton_method "f=x^2 - 2" n=3 x0=1
x_n = 1.414215686
f_x_n = 6.007304883e-06
iterates = [1.5,1.4166666666666667,1.4142156862745099]
  by: Newton's method  (Newton-Raphson with the exact derivative, at 30 digits)
  x0 = 1   []
  f = x^2 - 2
  assumes: f'(x_k) != 0 at every iterate; 1 <= n <= 50
```

What the worked example means: Newton's method computes successive approximations to a root of the function f, starting from the initial guess x0. Using the exact derivative at thirty digits of precision, the algorithm updates the estimate by subtracting the ratio of the function value to its derivative. After performing 3 iterations, the final approximation for the root is 1.41422, and the residual error at this point is given by f evaluated at that root, which equals 6.0073e-06. This process relies on the tangent line intersecting the horizontal axis to guide each step toward the solution.

Parametric curves (2)

Arc length of a parametric curve — `parametric_arc_length`

The length of the curve x = x(t), y = y(t) for t0 <= t <= t1.

$$L = \int_{t_0}^{t_1} \sqrt{x'(t)^2 + y'(t)^2}\, dt$$

symbol		meaning	SI unit
`xt`	in	x(t)	—
`yt`	in	y(t)	—
`t0`	in	start parameter	—
`t1`	in	end parameter	—
`arc_length`	out	the length of the curve	—

Applies when: t0 < t1; x and y continuously differentiable.

```
$ calculus solve parametric_arc_length t0=0 t1=3.141592654 "xt=cos(t)" "yt=sin(t)"
arc_length = 3.141592654
  by: Arc length of a parametric curve  (sqrt(x'(t)^2 + y'(t)^2) with exact derivatives, tanh-sinh
quadrature)
  t0 = 0   []
  t1 = 3.141592654   []
  xt = cos(t)
  yt = sin(t)
  assumes: t0 < t1; x and y continuously differentiable
```

What the worked example means: This calculation determines the geometric length of a curve defined by parametric equations x(t) and y(t). It integrates the speed of traversal along the path from the start parameter 0 to the end parameter 3.14159. The resulting value, 3.14159, represents the actual distance traveled along the trajectory between these two points.

Slope and concavity of a parametric curve — `parametric_slope`

dy/dx and d^2y/dx^2 of the curve x = x(t), y = y(t) at a parameter value t0.

$$\frac{dy}{dx} = \frac{y'(t)}{x'(t)}, \quad \frac{d^2y}{dx^2} = \frac{\frac{d}{dt}\frac{dy}{dx}}{x'(t)}$$

symbol		meaning	SI unit
`xt`	in	x(t)	—
`yt`	in	y(t)	—
`t0`	in	the parameter value	—
`dy_dx`	out	dy/dx at t0	—
`d2y_dx2`	out	d^2y/dx^2 at t0	—

Applies when: dx/dt != 0 at t0.

Not in `calculus` 1.1.0 yet: use the Python package or the model's page in the browser.

What the worked example means: This calculation determines the instantaneous rate of change of y with respect to x, known as the slope, which equals 1.5, and the rate of change of that slope, or concavity, which equals 0.75. These values are derived from the parametric functions for position at the specific time 1, providing insight into the curve's geometry and motion direction without requiring an explicit function of x.

Polar curves (1)

Area in polar coordinates — `polar_area`

The area swept by the polar curve r = r(theta) between theta = alpha and theta = beta.

$$A = \frac{1}{2}\int_{\alpha}^{\beta} r(\theta)^2\, d\theta$$

symbol		meaning	SI unit
`r`	in	the polar curve r(theta)	—
`al`	in	alpha, the start angle (radians)	—
`be`	in	beta, the end angle (radians)	—
`area`	out	the area	—

Applies when: alpha < beta <= alpha + 2 pi; r continuous.

```
$ calculus solve polar_area al=0 be=6.283185307 "r=1 + cos(theta)"
area = 4.71238898
  by: Area in polar coordinates  ((1/2) integral of r^2 by tanh-sinh quadrature)
  al = 0   []
  be = 6.283185307   []
  r = 1 + cos(theta)
  assumes: alpha < beta <= alpha + 2 pi; r continuous
```

What the worked example means: This calculation determines the area enclosed by the polar curve between the start angle and the end angle. Integrating half of the square of the radius over this angular interval shows that the resulting area is 4.71239. This value represents the total region covered by the curve as it rotates from 0 to 6.28319.

Series (6)

Partial sum of a geometric series — `geometric_partial_sum`

The sum of the first n terms a + a r + ... + a r^(n-1).

$$S_n = a\,\frac{1 - r^n}{1 - r}$$

symbol		meaning	SI unit
`a`	in	the first term	—
`r`	in	the common ratio	—
`n`	in	the number of terms	—
`sum`	out	the sum of the first n terms	—

Applies when: 1 <= n <= 500.

```
$ calculus solve geometric_partial_sum a=2 n=4 r=3
sum = 80
  by: Partial sum of a geometric series  (a (1 - r^n)/(1 - r) (n a when r = 1), exact in the stated
decimals)
  a = 2   []
  r = 3   []
  assumes: 1 <= n <= 500
```

What the worked example means: This calculation determines the total accumulation of the first 4 terms in a sequence where each term grows by multiplying the previous one by a constant factor. Starting with an initial value of 2, the sequence progresses through 4 steps using a multiplier of 3. The resulting sum, 80,

represents the combined magnitude of all these individual terms added together, providing a precise snapshot of the total value reached at that specific stage of growth.

Ratio test — `ratio_test`

The limit L of |a_(n+1)/a_n| and the ratio test's verdict for the series sum a_n (converges if L < 1, diverges if L > 1).

$$L = \lim_{n \to \infty} \left| \frac{a_{n+1}}{a_n} \right|$$

symbol		meaning	SI unit
`a`	in	the general term a_n, in n	—
`ratio_limit`	out	L = lim |a_(n+1)/a_n|	—
`verdict`	out	converges or diverges	—

Applies when: L exists, is finite and is not 1 (L = 1 is inconclusive and refused).

Not in `calculus` 1.1.0 yet: use the Python package or the model's page in the browser.

What the worked example means: This calculation determines the convergence behavior of the series defined by the general term 2^n/n^3. Evaluating the limit of the absolute ratio between consecutive terms as n approaches infinity gives the value 2. Since this limit is strictly less than one, the ratio test concludes that the series converges absolutely. The result depends entirely on the asymptotic growth rate of the terms relative to each other.

Sum of an arithmetic series — `arithmetic_series_sum`

The sum of the first n terms of an arithmetic sequence, and its last term.

$$S_n = \frac{n}{2}(2a_1 + (n-1)d)$$

symbol		meaning	SI unit
`a1`	in	the first term	—
`d`	in	the common difference	—
`n`	in	the number of terms	—
`sum`	out	the sum of the first n terms	—
`last_term`	out	the n-th term	—

Applies when: n >= 1.

```
$ calculus solve arithmetic_series_sum a1=2 d=3 n=10
sum = 155
last_term = 29
  by: Sum of an arithmetic series  (S_n = n (2 a_1 + (n - 1) d)/2, exact in the stated decimals)
  a1 = 2   []
  d = 3   []
  assumes: n >= 1
```

What the worked example means: This calculation determines the total accumulation of a sequence where each step adds a fixed amount. Given a starting value of 2 and a constant difference of 3, it computes the sum of the first 10 terms to be 155. The final value in this series, which represents the 10th term, is calculated as 29. This method provides an exact result for linearly spaced data points within the defined range.

Sum of an infinite geometric series — `geometric_series_sum`

The sum a + a r + a r^2 + ... of a convergent geometric series.

$$\sum_{n=0}^{\infty} ar^n = \frac{a}{1-r}$$

symbol		meaning	SI unit
`a`	in	the first term	—
`r`	in	the common ratio	—
`sum`	out	the sum of the series	—

Applies when: |r| < 1.

```
$ calculus solve geometric_series_sum a=3 r=0.5
sum = 6
  by: Sum of an infinite geometric series  (a/(1 - r), exact in the stated decimals)
  a = 3   []
  r = 0.5   []
  assumes: |r| < 1
```

What the worked example means: This calculation finds the total sum of an infinite geometric series using the first term 3 and the common ratio 0.5. The result, 6, represents the finite limit that the partial sums approach as the number of terms increases indefinitely. This method applies only when the absolute value of the common ratio is strictly less than one, ensuring convergence.

Sum of an infinite series — `series_sum`

The exact sum of a convergent series sum_{n=k}^infinity a_n (telescoping, geometric-type, zeta values, alternating harmonic, ...).

$$\sum_{n=k}^{\infty} a_n$$

symbol		meaning	SI unit
`a`	in	the general term a_n, in n	—
`k`	in	the starting index	—
`sum`	out	the sum of the series	—

Applies when: the series converges; a closed form exists.

Run `calculus info series_sum` for its inputs, then `calculus solve series_sum name=value ...`.

What the worked example means: This calculation determines the precise closed-form value of an infinite series. Given the starting index 1 and the general term defined by 1/n^2, the method evaluates whether the series converges to a specific number. The resulting total sum is 1.64493. This approach relies on symbolic techniques such as telescoping or geometric series identification, ensuring that the answer is mathematically exact rather than a numerical approximation.

Taylor polynomial — `taylor_polynomial`

The degree-n Taylor polynomial of f about x = a (Maclaurin when a = 0): its coefficients, its value at x1, f(x1) and the truncation error.

$$P_n(x) = \sum_{k=0}^{n} \frac{f^{(k)}(a)}{k!}(x-a)^k$$

symbol		meaning	SI unit
`f`	in	the function f(x)	—
`a`	in	the centre x = a	—
`n`	in	the degree	—
`x1`	in	the evaluation point	—
`coefficients`	out	c_0 .. c_n with c_k = f^(k)(a)/k!	—
`value`	out	P_n(x1)	—
`function_value`	out	f(x1)	—
`error`	out	f(x1) - P_n(x1)	—

Applies when: f is n times differentiable at a and defined at x1; 0 <= n <= 12.

```
$ calculus solve taylor_polynomial a=0 "f=exp(x)" n=3 x1=0.5
coefficients = [1.0,1.0,0.5,0.16666666666666666]
value = 1.645833333
function_value = 1.648721271
error = 0.002887937367
  by: Taylor polynomial  (exact symbolic derivatives at a divided by k!, then Horner evaluation in
(x1 - a))
  a = 0   []
  x1 = 0.5   []
  f = exp(x)
  assumes: f is n times differentiable at a and defined at x1; 0 <= n <= 12
```

What the worked example means: This calculation determines the coefficients and value of the degree 3 Taylor polynomial for the function exp(x) centered at 0. The resulting polynomial approximation, evaluated at the point 0.5, gives the value 1.64583. This is compared against the true function value 1.64872 to quantify the truncation error 0.00288794, which represents the difference between the exact result and the polynomial estimate.

Special functions (8)

Bessel function of the first kind — `bessel_j`

The Bessel function J_n(x) of whole-number order n.

$$J_n(x) = \sum_{m=0}^{\infty} \frac{(-1)^m}{m!\,(m+n)!} \left(\frac{x}{2}\right)^{2m+n}$$

symbol		meaning	SI unit
`n`	in	the order n	—
`x`	in	the argument x	—
`J`	out	J_n(x)	—

Applies when: 0 <= n <= 50; |x| <= 1000.

```
$ calculus solve bessel_j n=0 x=1
J = 0.7651976866
  by: Bessel function of the first kind  (hypergeometric series at 30 digits)
  x = 1   []
  assumes: 0 <= n <= 50; |x| <= 1000
```

What the worked example means: This calculation determines the value of the Bessel function of the first kind, denoted as J, using a hypergeometric series method accurate to thirty digits. The result depends on the integer order n and the real argument x. For the given inputs of order 0 and argument 1, the computed output is 0.765198. This function models oscillatory behavior in cylindrical systems, such as wave propagation or heat distribution within a cylinder.

Beta function — `beta_function`

The beta function B(a, b) for positive a and b.

$$B(a,b) = \int_0^1 t^{a-1}(1-t)^{b-1}dt = \frac{\Gamma(a)\Gamma(b)}{\Gamma(a+b)}$$

symbol		meaning	SI unit
`a`	in	the first argument (> 0)	—
`b`	in	the second argument (> 0)	—
`beta`	out	B(a, b)	—

Applies when: a > 0; b > 0.

```
$ calculus solve beta_function a=2 b=3
beta = 0.08333333333
  by: Beta function  (exp(ln Gamma(a) + ln Gamma(b) - ln Gamma(a + b)))
  a = 2   []
  b = 3   []
  assumes: a > 0; b > 0
```

What the worked example means: This calculation determines the beta function B(a, b) using inputs a set to 2 and b set to 3. The result, 0.0833333, represents the integral of t raised to the power of a minus one multiplied by one minus t raised to the power of b minus one over the interval from zero to one. This value is fundamental in probability theory, particularly for defining distributions on bounded intervals.

Chebyshev polynomial — `chebyshev_polynomial`

The value of the Chebyshev polynomial of the first kind T_n(x) (T_n(cos t) = cos(n t)).

$$T_{k+1}(x) = 2x\,T_k(x) - T_{k-1}(x)$$

symbol		meaning	SI unit
`n`	in	the degree n	—
`x`	in	the argument x	—
`T`	out	T_n(x)	—

Applies when: 0 <= n <= 500.

```
$ calculus solve chebyshev_polynomial n=3 x=0.5
T = -1
  by: Chebyshev polynomial  (the recurrence T_(k+1) = 2x T_k - T_(k-1))
  x = 0.5   []
  assumes: 0 <= n <= 500
```

What the worked example means: This calculation determines the value of the Chebyshev polynomial of the first kind using the recurrence relation. Given the degree 3 and the input argument 0.5, the resulting polynomial value is -1. This function oscillates between negative one and one when the argument lies within that interval, which makes it valuable for minimizing maximum error in approximation tasks.

Error function — `error_function`

The error function erf(x) and the complementary error function erfc(x) = 1 - erf(x).

$$\operatorname{erf}(x) = \frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2}\,dt$$

symbol		meaning	SI unit
`x`	in	the argument x	—
`erf`	out	erf(x)	—
`erfc`	out	erfc(x)	—

Applies when: x real.

```
$ calculus solve error_function x=0.5
erf = 0.5204998778
erfc = 0.4795001222
  by: Error function  (the standard library's erf and erfc (double precision))
  x = 0.5   []
  assumes: x real
```

What the worked example means: The calculation determines the error function value for the real argument 0.5. This result, 0.5205, represents the normalized area under a Gaussian curve from zero to that point. The complementary value, 0.4795, gives the remaining tail probability such that the two outputs sum to one.

Gamma function — `gamma_function`

The value of the gamma function (Gamma(n) = (n-1)! for whole n).

$$\Gamma(x) = \int_0^\infty t^{x-1} e^{-t}\,dt$$

symbol		meaning	SI unit
`x`	in	the argument x	—
`gamma`	out	Gamma(x)	—

Applies when: x is not 0 or a negative whole number; x < 171.6.

```
$ calculus solve gamma_function x=4.5
gamma = 11.6317284
  by: Gamma function  (the standard library's Lanczos-type approximation (double precision))
  x = 4.5   []
  assumes: x is not 0 or a negative whole number; x < 171.6
```

What the worked example means: This calculation determines the value of the gamma function at the input argument 4.5, yielding the result 11.6317. The gamma function extends the concept of factorial to real numbers, serving as a fundamental tool in probability and statistics. For whole number inputs, this value corresponds to the factorial of the preceding integer, effectively bridging discrete combinatorics with continuous mathematical analysis.

Legendre polynomial — `legendre_polynomial`

The value of the Legendre polynomial P_n(x).

$$(k+1)P_{k+1}(x) = (2k+1)\,x\,P_k(x) - k\,P_{k-1}(x)$$

symbol		meaning	SI unit
`n`	in	the degree n	—
`x`	in	the argument x	—
`P`	out	P_n(x)	—

Applies when: 0 <= n <= 200.

```
$ calculus solve legendre_polynomial n=3 x=0.5
P = -0.4375
  by: Legendre polynomial  (Bonnet's three-term recurrence (k+1) P_(k+1) = (2k+1) x P_k - k P_(k-
1))
  x = 0.5   []
  assumes: 0 <= n <= 200
```

In the worked example, n = 3, x = 0.5 go through the equation above and give P = -0.4375. It applies only when 0 <= n <= 200: outside that it says what is wrong instead of returning a number.

Regularized incomplete gamma functions — `regularized_gamma`

The regularized lower and upper incomplete gamma functions P(a, x) and Q(a, x) = 1 - P(a, x).

$$P(a,x) = \frac{1}{\Gamma(a)} \int_0^x t^{a-1} e^{-t}\, dt$$

symbol		meaning	SI unit
`a`	in	the shape a (> 0)	—
`x`	in	the upper limit x (>= 0)	—
`P`	out	P(a, x) = gamma(a, x)/Gamma(a)	—
`Q`	out	Q(a, x) = Gamma(a, x)/Gamma(a)	—

Applies when: a > 0; x >= 0.

```
$ calculus solve regularized_gamma a=2 x=3
P = 0.8008517265
Q = 0.1991482735
  by: Regularized incomplete gamma functions  (power series for x < a + 1, Lentz continued fraction
otherwise (the smaller of P and Q computed directly))
  a = 2   []
  x = 3   []
  assumes: a > 0; x >= 0
```

What the worked example means: This calculation determines the normalized cumulative probability up to a specific point for a gamma distribution. Using the shape parameter a = 2 and the integration limit x = 3, the algorithm evaluates the integral of the power-law weighted exponential decay. The result, P = 0.800852, represents the proportion of the total area under the curve from zero to x, while its complement, Q = 0.199148, accounts for the remaining tail area beyond that point.

Riemann zeta function — `riemann_zeta`

The Riemann zeta function zeta(s) = sum 1/n^s for s > 1 (the p-series sum).

$$\zeta(s) = \sum_{n=1}^{\infty} \frac{1}{n^s}$$

symbol		meaning	SI unit
`s`	in	the argument s (> 1)	—
`zeta`	out	zeta(s)	—

Applies when: s > 1.

```
$ calculus solve riemann_zeta s=2
zeta = 1.644934067
  by: Riemann zeta function  (Euler-Maclaurin / Borwein evaluation at 30 digits)
  s = 2   []
  assumes: s > 1
```

What the worked example means: This calculation determines the value of the Riemann zeta function, which is defined as the infinite sum of the reciprocals of integer powers. For the input argument 2, the computed output is 1.64493. This series converges only when the argument is strictly greater than one, representing a fundamental constant in number theory and physics.

Vector calculus (7)

Curl — `curl`

The curl of a vector field F = (P, Q, R) at a point.

$$\nabla \times \mathbf{F} = (R_y - Q_z,\ P_z - R_x,\ Q_x - P_y)$$

symbol		meaning	SI unit
`P`	in	the x-component P(x, y, z)	—
`Q`	in	the y-component Q(x, y, z)	—
`R`	in	the z-component R(x, y, z)	—
`p`	in	the point (x, y, z)	—
`curl`	out	curl F at the point	—

Applies when: the components are differentiable at the point.

```
$ calculus solve curl "P=x*y" "Q=y*z" "R=z*x" "p=[1, 2, 3]"
curl = [-2.0,-3.0,-1.0]
  by: Curl  (exact symbolic partial derivatives: (R_y - Q_z, P_z - R_x, Q_x - P_y))
  Q = y*z
  R = z*x
  P = x*y
  assumes: the components are differentiable at the point
```

What the worked example means: This calculation determines the rotation or circulation density of the vector field defined by components P, Q, and R. Evaluating the exact symbolic partial derivatives at the specific location [1, 2, 3] gives the result [-2, -3, -1]. This value indicates the tendency of the field to swirl around that point, which is essential for analyzing rotational motion in fluids and magnetic fields.

Divergence — `divergence`

The divergence P_x + Q_y + R_z of a vector field F = (P, Q, R) at a point.

$$\nabla \cdot \mathbf{F} = P_x + Q_y + R_z$$

symbol		meaning	SI unit
`P`	in	the x-component P(x, y, z)	—
`Q`	in	the y-component Q(x, y, z)	—
`R`	in	the z-component R(x, y, z)	—
`p`	in	the point (x, y, z)	—
`divergence`	out	div F at the point	—

Applies when: the components are differentiable at the point.

```
$ calculus solve divergence "P=x*y" "Q=y*z" "R=x*z" "p=[1, 2, 3]"
divergence = 6
  by: Divergence  (exact symbolic partial derivatives)
  Q = y*z
  R = x*z
  P = x*y
  assumes: the components are differentiable at the point
```

What the worked example means: The divergence is calculated by summing the partial derivatives of the vector field components with respect to their corresponding coordinates. Using the input values for P, Q, and R at the specified point p shows that the divergence is 6. This result quantifies the extent to which the point acts as a source or sink for the flow described by the field.

Green's theorem on a rectangle — `greens_theorem`

Both sides of Green's theorem for F = (P, Q) on the rectangle [a, b] x [c, d]: the counter-clockwise circulation around the boundary and the double integral of Q_x - P_y.

$$\oint_C P\,dx + Q\,dy = \iint_R (Q_x - P_y)\,dA$$

symbol		meaning	SI unit
`P`	in	the x-component P(x, y)	—
`Q`	in	the y-component Q(x, y)	—
`a`	in	x from	—
`b`	in	x to	—
`c`	in	y from	—
`d`	in	y to	—
`circulation`	out	the boundary line integral (counter-clockwise)	—
`double_integral`	out	the integral of Q_x - P_y	—

Applies when: a < b, c < d; P and Q continuously differentiable on the rectangle.

Not in `calculus` 1.1.0 yet: use the Python package or the model's page in the browser.

What the worked example means: This calculation verifies Green's theorem by independently computing the counter-clockwise line integral around the boundary of the rectangle and the double integral of the curl, defined as the partial derivative of Q with respect to x minus the partial derivative of P with respect to y. The result shows that the total circulation 4 matches the accumulated rotation within the region, represented by the double integral value 4. This agreement holds provided the vector field components are continuously differentiable across the entire domain defined by the input bounds.

Laplacian — `laplacian`

The Laplacian f_xx + f_yy (+ f_zz) of a function at a point.

$$\nabla^2 f = f_{xx} + f_{yy} + f_{zz}$$

symbol		meaning	SI unit
`f`	in	the function f(x, y) or f(x, y, z)	—
`p`	in	the point	—
`laplacian`	out	the Laplacian at the point	—

Applies when: f twice differentiable at the point.

```
$ calculus solve laplacian "f=x^2*y + z^3" "p=[1, 2, 3]"
laplacian = 22
  by: Laplacian  (exact second partial derivatives summed)
  f = x^2*y + z^3
  assumes: f twice differentiable at the point
```

What the worked example means: This calculation determines the Laplacian by summing the unmixed second partial derivatives of the function with respect to each spatial variable. The result, 22, quantifies the difference between the function's value at the specific point and its average over a small surrounding sphere, which is fundamental in analyzing diffusion and potential fields.

Line integral of a vector field — `line_integral_vector`

The line integral (work) of F = (P, Q) along the path x = x(t), y = y(t) from t0 to t1.

$$\int_C \mathbf{F} \cdot d\mathbf{r} = \int_{t_0}^{t_1} \big(P\,x'(t) + Q\,y'(t)\big)\,dt$$

symbol		meaning	SI unit
`P`	in	the x-component P(x, y)	—
`Q`	in	the y-component Q(x, y)	—
`xt`	in	the path x(t)	—
`yt`	in	the path y(t)	—
`t0`	in	start parameter	—
`t1`	in	end parameter	—

symbol		meaning	SI unit
`line_integral`	out	the integral of F . dr along the path	—

Applies when: the integrand is continuous for t between t0 and t1.

```
$ calculus solve line_integral_vector P=-y Q=x t0=0 t1=6.283185307 "xt=cos(t)" "yt=sin(t)"
line_integral = 6.283185307
  by: Line integral of a vector field  (F(r(t)) . r'(t) built exactly (symbolic r'), then tanh-sinh
quadrature at 20 digits)
  t0 = 0   []
  t1 = 6.283185307   []
  Q = x
  yt = sin(t)
  xt = cos(t)
  P = -y
  assumes: the integrand is continuous for t between t0 and t1
```

What the worked example means: This calculation determines the work done by the vector field defined by its x and y components as it moves along the parametric curve from the start parameter to the end parameter. By constructing the dot product of the field and the derivative of the path symbolically, the method integrates this quantity over the given interval. The resulting value of 6.28319 represents the total accumulation of the field's influence along that specific trajectory.

Line integral with respect to arc length — `line_integral_scalar`

The integral of f(x, y) ds along the path x = x(t), y = y(t), t0 <= t <= t1 (mass of a wire, average along a curve).

$$\int_C f\,ds = \int_{t_0}^{t_1} f(x(t), y(t))\sqrt{x'(t)^2 + y'(t)^2}\,dt$$

symbol		meaning	SI unit
`f`	in	the function f(x, y)	—
`xt`	in	the path x(t)	—
`yt`	in	the path y(t)	—
`t0`	in	start parameter	—
`t1`	in	end parameter	—
`line_integral`	out	the integral of f ds along the path	—

Applies when: t0 < t1; the integrand continuous on [t0, t1].

Not in `calculus` 1.1.0 yet: use the Python package or the model's page in the browser.

What the worked example means: This calculation determines the total accumulation of the scalar function x + y along the curved path defined by x(t) and y(t). The process involves substituting the parametric equations into the function and multiplying by the magnitude of the derivative vector, which represents the arc length element. The result, 1.41421, quantifies physical properties such as the mass of a wire with variable density or the average value of a field along a trajectory. This method transforms a geometric integral into a standard single-variable definite integral over the parameter interval from 0 to 1.

Outward flux through a box (divergence theorem) — `divergence_theorem_box`

The outward flux of F = (P, Q, R) through the surface of the box [x0, x1] x [y0, y1] x [z0, z1].

$$\oiint_S \mathbf{F} \cdot \mathbf{n}\, dS = \iiint_B \nabla \cdot \mathbf{F}\, dV$$

symbol		meaning	SI unit
`P`	in	the x-component	—
`Q`	in	the y-component	—
`R`	in	the z-component	—
`x0`	in	x from	—
`x1`	in	x to	—
`y0`	in	y from	—
`y1`	in	y to	—
`z0`	in	z from	—
`z1`	in	z to	—
`flux`	out	the outward flux	—

Applies when: the box is non-degenerate; F continuously differentiable on the box.

```
$ calculus solve divergence_theorem_box "P=x^2" "Q=y^2" "R=z^2" x0=0 x1=1 y0=0 y1=1 z0=0 z1=1
flux = 3
  by: Outward flux through a box (divergence theorem)  (the divergence theorem: the triple integral
of the exact divergence (exact for polynomials, otherwise Gauss-Legendre product rules until two
orders agree))
  x0 = 0   []
  x1 = 1   []
  y0 = 0   []
  y1 = 1   []
  z0 = 0   []
  z1 = 1   []
  P = x^2
  Q = y^2
  R = z^2
  assumes: the box is non-degenerate; F continuously differentiable on the box
```

What the worked example means: This calculation determines the net flow of the vector field out of the rectangular region defined by the input coordinates. By applying the divergence theorem, the complex surface integral is converted into a simpler volume integral of the field's divergence over the box. The resulting value, 3, represents the total amount of the field leaving the enclosed space per unit time.

Questions and reports: support@ai999x.com. AI999X — The Future Is Self-Improving.